

1. Glyphs 4

Source: Daring Fireball | Published: 2026-09-13T21:44:17Z

Link: <https://glyphsapp.com/>

My thanks to Glyphs for sponsoring this last week at DF. Glyphs 4 is a truly native Mac app for creating fonts, lettering, icons, and pictograms. Glyphs lets you draw with flexible strokes and efficiently reuse shapes across huge sets of glyphs, and explore variable vector designs with complex higher-order interpolation. Glyphs lets you build and manage massive font family and icon sets with ease, and it exports all modern font formats, as well as SVG, PNG, and PDF. Earlier this year Glyph co-creators Georg Seifert and Rainer Scheichelbauer were awarded the RIT Frederic W. Goudy Award for their work on Glyphs. The list of previous winners of this prestigious award reads like a hall of fame for typography: Matthew Carter, Adrian Frutiger, Gudrun and Hermann Zapf, Robert Brinhurst, and Robert Slimbach, to name a few. Glyphs hits my personal interests hard. I'm deeply interested in and fascinated by typography, and nothing pleases me more than a Mac-assed Mac app. I am not, however, a typeface designer. Glyphs is so good, though, that it makes me want to be. This comment from one Glyphs user says a lot: "After using Glyphs, the pen tool in Illustrator feels like carving a chicken wi..."

2. Yours Truly on Off Protocol With Jim Ray

Source: Daring Fireball | Published: 2026-09-12T15:36:53Z

Link: <https://atproto.com/off-protocol/2026-09-11-make-your-own-thing-john-gruber>

My old friend Jim Ray now runs developer relations at Bluesky, and part of that gig is hosting a podcast, Off Protocol (a show about building a better Internet?). I was delighted to appear as his latest guest. I generally don't like talking about my career, but I did enjoy talking about it with Jim. He has that effect on people. ?

3. T-Mobile Is Charging \$5/Month for iPhone Handoff

Source: Daring Fireball | Published: 2026-09-12T15:24:09Z

Link: <https://www.t-mobile.com/news/devices/iphone-18-pro-iphone-duo-apple-watch-release>

T-Mobile, in their iPhone 18 Pro / iPhone Duo press release: Plus, T-Mobile is one of the first to support Apple's new iPhone Handoff feature, where customers with two compatible iPhone models can use their T-Mobile number across both devices and choose which phone is active at any given time. The secondary iPhone works seamlessly, including essential T-Mobile services like calls, texts, data, mobile hotspot, Scam Shield and T-Satellite on eligible plans, allowing customers to get the most from two devices without clunky handoffs or the hassle of managing another number. iPhone Handoff is available on any iPhone with iOS 27 via iPhone settings and ready to use from day one on the new iPhone 18 Pro lineup and iPhone Duo. T-Mobile is introducing the feature at \$5/month. This seems like a ridiculous thing to charge for. It's not adding a separate device to your plan, like adding a cellular iPad or even an Apple Watch. It's just switching your active eSIM between two iPhones. You can do this for free, now, on any carrier via Settings > Cellular > but it takes a bunch of taps, confirmation on both iPhones, and a few minutes for the carrier to register the eSIM transfer. iPhone Handoff...

4. commit-rewriter 0.1

Source: Simon Willison's Weblog | Published: 2026-09-14T00:28:10+00:00

Link: <https://simonwillison.net/2026/Sep/14/commit-rewriter/>

Release: commit-rewriter 0.1 I built this little web app the other day to help edit the commit messages for the Datasette security releases . The initial commits were full of coding agent cruft and references to issue IDs from our private repository, so they weren't fit for publication. If you want to edit the commit messages for a repository you can run it like this: `uvx commit-rewriter path/to/repo` Omit the path if you are already in the directory for that repo. When you submit your edits the tool creates a timestamped branch of your current repo state - to allow you to revert if you need to - and then rewrites every commit from the first

one you edited to the most recent. Tags: git , projects , python , ai-assisted-programming

5. shot-scraper 1.12

Source: Simon Willison's Weblog | Published: 2026-09-13T23:58:14+00:00

Link: <https://simonwillison.net/2026/Sep/13/shot-scraper/>

Release: shot-scraper 1.12 I've added WebP support to my shot-scraper screenshot automation tool. You can now take a WebP screenshot of a web page like this: shot-scraper

`https://simonwillison.net -o screenshot.webp --quality 80` The `--quality` option sets the quality - without that option the WebP file will be lossless. In my experience WebP screenshots are almost always significantly smaller in file size than their JPEG or PNG equivalents. See the PR for some examples. I shipped this feature so I could use it to generate the screenshot for my new commit-rewriter tool . Tags: playwright , shot-scraper

6. Feeling sad about AI

Source: Simon Willison's Weblog | Published: 2026-09-11T17:28:37+00:00

Link: <https://simonwillison.net/2026/Sep/11/feeling-sad-about-ai/>

My comment on Feeling sad about AI ? Hacker News. I'm not sure how useful it is to say this, but I think a lot of people (myself included, a few years ago now) have been through this moment of existential crisis and come out the other side. The initial reaction, when some coding agent does a piece of work that would have taken you a week in an hour and does it well , is to be very disheartened by it. Once you come to terms with the idea that translating an exact specification into decent code isn't a unique skill any more, you can start looking at the larger set of problems that you face as a software engineer and realize that there is so much left , and your existing skill and experience mean you can master these new tools, provide value, and execute at a level far greater than anyone who is just getting started building software using agents without any of your depth. If you don't want your profession to change at all then you're going to have a tough time with this - but that's surely been true for the history of software engineering? Has there ever been any stability to the tools and language we use beyond about a five year time horizon? These changes are happening a bit faster...

7. Slow developer experience will bottleneck fast models

Source: seangoedecke.com RSS feed | Published: Mon, 14 Sep 2026 00:00:00 GMT

Link: <https://seangoedecke.com/slow-devex-will-bottleneck-fast-models/>

Right now developer experience is measured in seconds. If your tests take a second to run, that's good; if they take thirty seconds, that's bad. Any faster than a second doesn't really matter, because most of your time is spent either thinking or waiting for an AI agent to spin. Shaving milliseconds off your dev server reload time or whatever is pointless: that's not the bottleneck. It will be. Small models are getting faster and faster, and smart models are getting smaller. I think most engineers will still want to use the smartest available model ? software engineering is hard ? but we will increasingly see faster models get used as subagents or for well-understood tasks. This is largely uncharted territory. Very few people have developed intuitions for what it is going to be like to work with agents that run at thousands of tokens-per-second. GPT-6-Astra can run at about sixty tokens per second. That means you spend a lot of time waiting for it to think. You work with it like you would work with another human: delegating a task and then context-switching until that task is complete. If you haven't yet, have a play around with Jimmy , Taalas' version of LLaMA-3.1-8B running 1 at...

8. AI Forces You to Commit to Your Initial Belief

Source: Software and Tech stories from an Insider - iDiallo.com | Published: Sun, 13 Sep 2026 20:00:00 GMT

Link: <https://idiallo.com/blog/making-changes-mid-sentence>

Have you ever watched any old hacker movie lately? The hacker can write at 180 WPM, never using the backspace or changing his mind ever. It's like before they ever started, they already knew the last character they would type on the screen. But that's not how it works in real life, at least not for me. The idea changes mid-sentence. That's probably how most of the code I write

ends up. I start with an idea in my mind. I think it's brilliant, maybe it's a way to restructure the code, an overall design, or a specific paradigm. But when I start writing it, something else pops up. Seeing it halfway down the page, I realize it's not quite what I was imagining, and I change direction. The same principle applies when I'm writing a blog post or a story. I feel like I have a perfect concept in my head, but the moment I start putting words down, things shift and I adjust accordingly. That's why the final text rarely matches the original vision. It's as if writing itself is the process of refining the idea. This process is lost when you use AI. You don't write out the details of an idea as you go; instead, you just give it the gist of it. It generates an entire wall of text instantly, and no...

9. Anecdotally, programmers dislike "reduce"

Source: Evan Hahn (dot com) | Published: Sun, 13 Sep 2026 00:00:00 +0000

Link: <https://evanhahn.com/posts/2026-09-13-programmers-dislike-reduce/>

In short: from my experience, people like map and filter, but not reduce. I use functions like map and filter all the time. When I put that code up for review, my peers rarely complain. I get plenty of feedback about other decisions, but not about my use of map and filter. I cannot say the same for reduce. Often, when I've submitted a patch with reduce inside, I get a comment like, "this part is hard to read." And I see reduce way less than map, filter, some, and so on. Anecdotally, I have come to believe that programmers don't like reduce as much. I don't know why, but I have a few theories: reduce is harder to read. reduce is less familiar. reduce can have worse performance compared to other options. reduce is less elegant in languages I use, like JavaScript, Python, and Swift. In my blissful stint as a Clojure developer, I did not get this feedback. I'm wrong, and I'm seeing a trend that's not real. I usually just change reduce to something else and move on. Even though I prefer it, I don't usually care much. But it's a little social phenomenon I've observed, and I thought I'd document it. I've also noticed this less recently, possibly because code review is less thorough...

10. Preserving Netlify Analytics Data Beyond 30 Days

Source: Jim Nielsen's Blog | Published: Sun, 13 Sep 2026 19:00:00 GMT

Link: <https://blog.jim-nielsen.com/2026/preserving-netlify-analytics-data/>

I've got Netlify Analytics turned on for a few of my websites. I use it send myself a little daily digest that alerts me when folks link to my writing, be it an individual sending me a few referrals, a newsletter sending me a few hundred, or Hacker News sending me thousands. But sometimes I want to know more than, "What happened with traffic in the last 24 hours?" Sometimes I want to know, "What happened with traffic in the last month?" I can log in to Netlify to answer that question. Sometimes I want to know, "What happened with traffic in the last six months?" Netlify doesn't have an answer for that because they only store the last thirty days of traffic data. I've known about this limitation for years and always thought, "I really should just start saving that data for myself." But, you know, laziness. How bad do I really want it? The answer, it seems, was "I only want it if it's _easy_". Enter Val Town. Val Town makes this kind of task easy. Each Val is its own little bucket of scoped computing primitives for just such a task: Code (run a script that fetches data) Secrets (securely store my Netlify access token) Storage (my own little SQLite database for persisting data ov...

11. Fixing an NZXT Signal 4K30 part 2: the green/pink video bug

Source: Downtown Doug Brown | Published: Sun, 13 Sep 2026 21:02:59 +0000

Link: <https://www.downtowndougbrown.com/2026/09/fixing-an-nzxt-signal-4k30-part-2-the-green-pink-video-bug/>

Last year, I repaired an NZXT Signal 4K30 USB capture device that I bought on eBay for cheap. It was completely dead, and the cause turned out to be a bad solder joint on an inductor, which prevented power from getting to a crucial part of the board. After I fixed it, I tried using [?]